PLATEFORME DE TRAITEMENT DES PAIEMENTS

SCHEMA DE DONNEES DU SYSTEME NoSQL

Nature du document	
Version	**1.0**
Client	**Étude de cas – STRIPE**
Date de la version	**08/10/2026**
Description de la version	**Bloc 2 - certification RNCP41993**
Modules concernés	**Livrable 3**
Auteur	**Ronan GUILLOU**

Table des matières

1 Stratégie de relation 3

1.1 Définitions 3

2 Collections 3

2.1 Définitions 3

2.2 sessions 3

2.3 click_events 4

2.4 logs 4

2.5 customer_feedback 4

2.6 ml_features 5

2.7 inbound_payloads 5

3 Compteurs Redis 5

3.1 Contexte 5

3.2 Compteurs retenus 5

Note sur l'usage de l'intelligence artificielle

Pour rédiger ce document, j'ai utilisé un assistant d'IA générative comme outil d'aide à la rédaction. Il est intervenu sur quatre points : la correction de l'orthographe et de la syntaxe, la proposition de structures et de brouillons de texte que j'ai relus et modifiés, la comparaison d'idées et d'options avant décision, et la vérification de la cohérence d'ensemble.

Les choix retenus, la sélection des besoins et la validation finale de chaque chapitre relèvent de ma responsabilité.

1 Stratégie de relation

1.1 Définitions

Dans une base NoSQL, il existe plusieurs façons de relier les documents entre eux : les **intégrer** dans un même document (embedding), les **référencer** par un identifiant (referencing), et les **indexer** pour accélérer les recherches (indexing). Le choix dépend du volume, de la fréquence de lecture et de l'intégration avec les systèmes OLTP et OLAP.

Le tableau ci-après détail les différentes stratégie de relation possible pour un document stocké :

Relation	Principe	Explication
Embedding (intégrer)	Les données liées sont dans le même document	Une seule lecture suffit, sans jointure : adapté aux petites données lues ensemble.
Référence	Le document garde l'identifiant d'un autre document	Évite la duplication et la croissance du document, au prix d'une seconde lecture.
Tableau intégré	Une liste d'éléments dans le document	Simple, mais seulement si la liste reste petite et bornée.
Subset pattern	On intègre les derniers éléments, le reste est en collection séparée	Lecture rapide de l'essentiel, historique complet conservé à part.
Valeur pré-calculée	On stocke le résultat (un total) plutôt que le détail	Évite de recalculer à chaque lecture.
Duplication volontaire	On copie un champ d'un autre système	Évite une jointure sur une donnée qui change peu, mais exige de la garder à jour.

2 Collections

Dans notre business case, nous partons sur 6 collections de documents. Toutes sont alimentées par l'API de collecte. Leurs données alimentent le pipeline (Bronze → Silver → Gold) pour les features froides, et les compteurs chauds de Redis sont mis à jour en direct pour la détection de fraude.

2.1 Définitions

Collection	Désignation	Source	Stratégie
sessions	Visite d'un client (appareil, IP hachée, résumé de navigation…)	Sites et applications	Embedding
click_events	Clics et pages vues d'une session. Pointe vers sa session_id.	Sites et applications	Référence
logs	Journaux d'accès et d'erreurs de la plateforme	Sites et applications	Embedding
customer_feedback	Avis et réponses d'enquêtes, en texte libre. L'avis est rattaché au client, au marchand et au paiement par leurs identifiants.	Plateforme	Référence
ml_features	Signaux bruts de comportement collectés côté client (saisie, écran, langue), entrée du calcul des features.	Clients	Référence
inbound_payloads	Messages JSON ou XML reçus tels quels, fichiers binaires dans S3. Ici pour indiquer qu'on pourrait stocker d'autres type de fichiers.	Sites et application	Embedding

2.2 sessions

Une visite d'un client sur le site ou l'application du marchand.

```
{
  "session_id": "ses_8f3a21c9",
  "device_id": "dev_a91c4e77",
  "customer_id": 1204,
  "merchant_id": 17,
  "ip_hash": "5e2b9f...c41a",
  "ip_country_id": 75,
  "device": {
```

```
    "type": "mobile",
    "os": "iOS 17",
    "browser": "Safari",
    "language": "fr-FR"
  },
  "stats": {
    "nb_pages": 6,
    "nb_clics": 14,
    "duration_s": 212
  },
  "started_at": "2026-10-08T15:02:11Z",
  "last_activity_at": "2026-10-08T15:05:43Z",
  "expires_at": "2027-01-06T15:02:11Z"
}
```


2.3 click_events

Un clic ou une page vue, rattaché à sa session.


```
{
  "event_id": "clk_00391827",
  "session_id": "ses_8f3a21c9",
  "customer_id": 1204,
  "merchant_id": 17,
  "product_id": 5532,
  "event_type": "click",
  "context": {
    "page": "/checkout",
    "element": "btn_pay",
    "position": {
      "x": 182,
      "y": 640
    }
  },
  "occurred_at": "2026-10-08T15:05:21Z",
  "expires_at": "2027-01-06T15:05:21Z"
}
```


2.4 logs

Une ligne de journal d'accès ou d'erreur de la plateforme.


```
{
  "log_id": "log_7721a0",
  "log_type": "error",
  "level": "ERROR",
  "merchant_id": 17,
  "transaction_id": 98234117,
  "context": {
    "service": "api-paiement",
    "message": "Timeout lors de l'appel à la banque",
    "http_status": 504,
    "duration_ms": 3012
  },
  "created_at": "2026-10-08T15:05:44Z",
  "expires_at": "2027-01-06T15:05:44Z"
}
```


2.5 customer_feedback

Un avis client en texte libre.


```
{
  "feedback_id": "fbk_20431",
  "customer_id": 1204,
  "merchant_id": 17,
  "transaction_id": 98234117,
  "content": {
    "rating": 2,
```

```
    "text": "Le paiement a échoué deux fois avant de passer.",
    "tags": [
      "paiement",
      "échec"
    ],
    "language": "fr"
  },
  "created_at": "2026-10-08T16:20:09Z"
}
```


2.6 ml_features

Signaux bruts de comportement collectés côté client, entrée du calcul des features.


```
{
  "signal_id": "sig_550912",
  "session_id": "ses_8f3a21c9",
  "customer_id": 1204,
  "signals": {
    "typing_speed_cpm": 212,
    "paste_detected": false,
    "screen_resolution": "390x844",
    "timezone": "Europe/Paris",
    "language": "fr-FR",
    "touch_support": true
  },
  "collected_at": "2026-10-08T15:05:20Z",
  "expires_at": "2027-01-06T15:05:20Z"
}
```


2.7 inbound_payloads

Un message reçu tel quel (ici en XML), avec la référence d'un fichier stocké dans S3.


```
{
  "payload_id": "pld_99120",
  "merchant_id": 17,
  "format": "xml",
  "payload": "<order><id>CMD-5521</id><amount currency=\"EUR\">49.90</amount></order>",
  "attachment_s3_path": "s3://stripe-inbound/17/2026-10-08/CMD-5521.pdf",
  "received_at": "2026-10-08T15:05:12Z",
  "schema_version": 1
}
```


3 Compteurs Redis

3.1 Contexte

Redis (AWS ElastiCache) sert de serveur de **cache** pour la détection de fraude. Il garde en mémoire des compteurs sur le comportement récent des clients, des cartes, des adresses IP et des marchands. Le service de scoring les lit en moins d'une milliseconde, sans interroger les bases, ce qui permet de tenir la décision en moins de 100 ms. Chaque compteur porte une durée de vie (TTL) et ses clés sont pseudonymisées. Redis ne contient aucune donnée de référence : il peut être reconstruit à partir des données sources.

3.2 Compteurs retenus

Voici la liste des compteurs retenus pour alimenter le cache Redis (a titre d'exemple) :

#	Compteur	Fenêtre et structure	Origine
C1	Paiements par client	10 min, INCR + TTL	**OLTP** : transaction (customer_id, created_at)
C2	Cartes distinctes par client	24 h, HyperLogLog	**OLTP** : payment_method (fingerprint)

C3	Clients distincts par carte	24 h, HyperLogLog	**OLTP** : payment_method (fingerprint), transaction (customer_id)
C4	Montant cumulé par client et par devise	24 h, Sorted Set	**OLTP** : transaction (amount, currency_id)
C5	Paiements par adresse IP	5 min, INCR + TTL	**OLTP** : transaction (ip_address) ;
C6	Appareils distincts par client	24 h, Set	**OLTP** : transaction (device_id) ;
C7	Paiements par marchand	1 min, INCR + TTL	**OLTP** : transaction (merchant_id)
C8	Profil de session avant paiement.	TTL 24 h, Hash	**OLTP** : transaction (session_id) ; **collections** : sessions (stats), click_events, ml_features
Liste de blocage	IP interdites (plages)	Recherche d'appartenance	**OLTP** : ip_blocklist (ip_range)

- **INCR** : *Increment*, ajoute 1 à un compteur de façon atomique.
- **TTL** : *Time To Live*, durée de vie d'une clé avant suppression automatique.
- **HyperLogLog** : structure de comptage approximatif des valeurs distinctes, très économe en mémoire.
- **Set** : ensemble de valeurs uniques, sans doublon.
- **Sorted Set** : ensemble trié par score (ici l'heure), qui permet une fenêtre glissante exacte.
- **Hash** : table de plusieurs champs sous une même clé, comme une petite fiche.